Design + V&V Document

Version 1
Group 22
COMPSCI 4ZP6

Dr. Mehdi Moradi
April 1, 2026

Andre Menezes	meneza3@mcmaster.ca	400401293
Leiah Nay	nayl@mcmaster.ca	400362166
Martin Matlok	matlokm@mcmaster.ca	400369490
Nicholas Poulidis	poulidin@mcmaster.ca	400380146
Tony Lin	lint50@mcmaster.ca	400388309

Table of Contents

1. Introduction

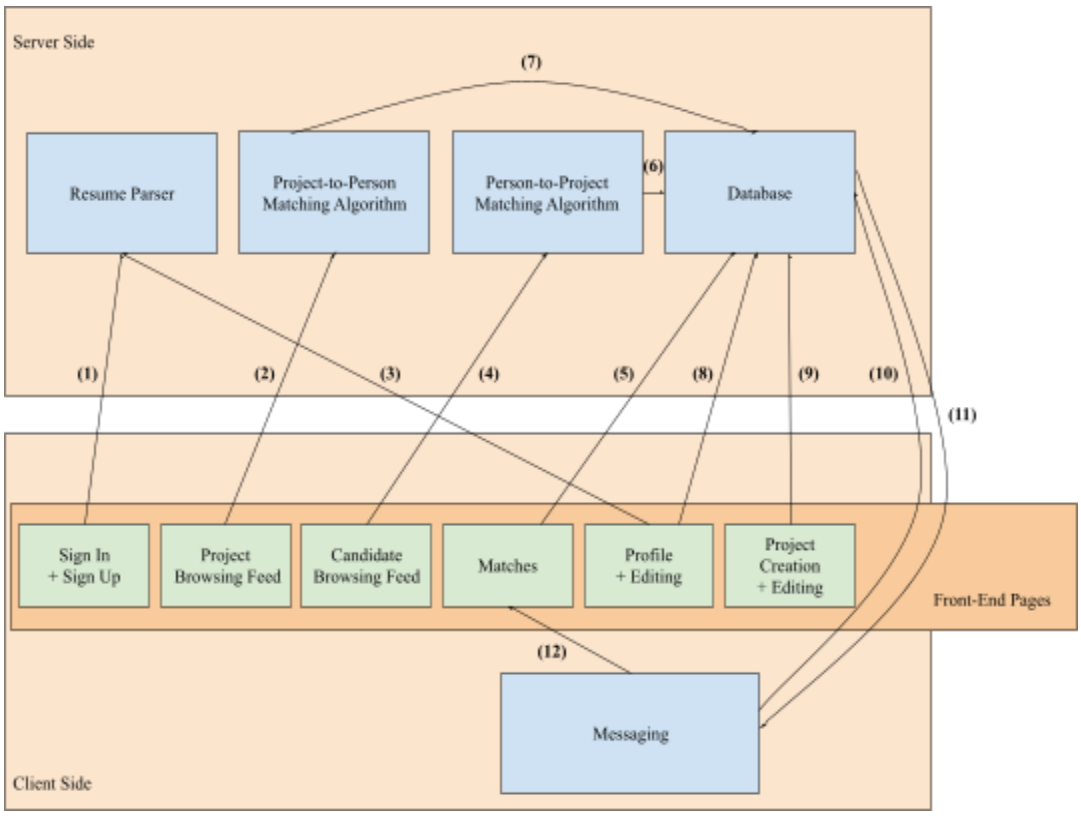
1.1 Project Purpose

Students and early professionals often seek opportunities to collaborate on projects outside of their coursework or workplace. However, finding reliable teammates or opportunities is often limited to personal networks, scattered online forums, and LinkedIn, making collaboration difficult. Without a central hub, individuals struggle to match their skills and interests with the right opportunities, leading to missed chances for learning, innovation, and community building. Peer.io is a mobile-based application where users can browse two interactive feeds: one for project ideas and one for potential collaborators. Using a like-dislike style interface, users can match with projects and peers. Once a mutual interest is established, contact information is exchanged to facilitate collaboration. Over time, the platform's matching algorithm will adapt to user preferences in combination with the user's skills and experience, making it easier to find meaningful and relevant opportunities.

1.2 Document Purpose

The purpose of this document is to describe, in detail, the components of our application and explain the relationship between each component, other component(s) and the requirements as defined in our Software Requirements Specification (SRS) document. We also provide an overview of our testing plan for each component to ensure we meet these requirements and provide a smooth and consistent user experience.

2. Project Component Diagram



1. Upon sign-up, the user is prompted to upload a resume. This resume is then parsed by the Resume Parser to initialize a user's profile.
2. The Project Browsing Feed displays possible projects in order of compatibility, as decided by the Project-to-Person Matching Algorithm.
3. While editing their profile, the user can choose to upload a new resume and have it parsed by the Resume Parser.
4. The Candidate Browsing Feed displays possible candidates to a project owner in order of compatibility, as decided by the Person-to-Project Matching Algorithm.
5. The matches page stores previous matches between candidates and projects as saved in the Database.
6. The Person-to-Project Matching Algorithm finds people that match a user's project(s). These possible candidates are chosen from the Database.
7. The Project-to-Person Matching Algorithm finds projects that match a user's profile. These possible projects are chosen from the Database.
8. A user can view and edit their profile. Any changes they make, will be updated in the Database.
9. A user can view and edit the projects they own. Any changes they make will be updated in the Database.
10. Whenever a user sends a message, that message will be stored in the Database.
11. When opening a previous chat, the user pulls previous messages from the Database.
12. Messaging chats can begin once a project and a candidate have a Match.

3. Relationship Between Project Components and Requirements

See *Appendix, Table 1*.

4. Project Components

4.1 Resume Parser

4.1.1 Normal Behaviour

The Resume Parser component is responsible for handling and processing a resume file submitted by the user. The user can either during account creation or from the profile page after account creation. The resume parser will then transform it into a structured representation which will be used for populating the user's profile. The document is then processed as a whole and converted into a standardized profile-oriented JSON structure.

The parser extracts key profile fields from the resume, including a short biography, location, professional and personal links, skills, interests, education history, work experience, and personal projects. Rather than relying on fixed section splitting alone, the parser interprets the document in context and maps relevant resume content into the appropriate output fields. It is designed to return only information clearly supported by the resume and to leave fields blank when information is missing or uncertain. The final result is also cleaned to remove duplicate entries, normalize links, and ensure consistent formatting before being returned to the application.

4.1.2 API

The parser is exposed through two API endpoints. The first endpoint accepts a file upload from the user, while the second accepts a JSON payload containing a URL to a remote resume file. In both cases, the service processes the provided document and returns a structured JSON response containing the parsed profile data.

The returned JSON includes fields such as bio, location, links, skills, interests, education, experience, and personal projects, along with parser metadata describing the source and parser version. This response format is designed so the mobile application can review, display, and save the extracted content directly into the user profile. If parsing cannot be completed successfully, such as when required configuration is missing, the document cannot be accessed, or the extracted output is invalid, the API returns an error response indicating that parsing failed.

4.1.3 Implementation

The Resume Parser is implemented as a service-oriented parsing workflow with a lightweight API layer and a dedicated extraction layer. The API layer handles incoming requests, validates the presence of a file or URL, and prepares the resume for processing. For uploaded files, the service temporarily stores the file before passing it into the parsing pipeline.

The extraction workflow processes the resume file and produces a schema-constrained JSON result representing the candidate's profile information. The extraction stage is guided by a predefined response structure that specifies which fields must be returned and how they should be organized. This helps ensure that the output remains consistent even when resume layouts are different. After the initial extraction step, the parser applies a normalization stage that cleans text fields, formats links into standard URL form where possible, removes duplicate content, filters out empty entries, and enforces reasonable limits on list-based fields such as skills, interests, education, experience, and projects.

The final output is then augmented with metadata, including the parser version, source type, and source name, and returned as a structured JSON object through the API.

4.1.4 Potential Undesired Behaviours

Most of the likely undesired behaviours arise from resume quality, formatting variation, or ambiguous content in the source document. If the resume is poorly formatted, image-heavy, difficult to read, or contains limited accessible text, extraction quality may decrease. Unusual layouts, nonstandard headings, dense visual formatting, or mixed ordering of content may cause some information to be assigned to the wrong field or omitted entirely.

The parser may also return incomplete results when the resume does not explicitly state certain information, since the service is designed to avoid inventing unsupported details. Skills, interests, project links, and location fields are particularly sensitive to whether the resume presents them clearly. In some cases, different roles or projects may be merged too aggressively during cleanup if they appear highly similar, while repeated or inconsistent formatting may still lead to partial duplication in edge cases. Lastly, larger files or remote files with slow accessibility may increase processing time, and invalid URLs or unsupported document inputs may cause the request to fail before parsing is completed.

4.2 Project-to-Person Matching Algorithm

4.2.1 Normal Behaviour

The project-to-person matching algorithm is the key factor in helping facilitate high-quality collaborative connections by ranking potential matches based on a weighted system. This algorithm helps enable project owners to find the most qualified candidates for their specific needs. Under normal operation, the algorithm computes a composite score for each potential candidate a project owner might want to invite to collaborate on their project with them. The score is calculated using three weighted components namely semantic similarity (weighted 0.45), skill match (weighted 0.50), and interest alignment (weighted 0.05). These weights were chosen through AUC testing of the algorithm which used LLM-as-judge justification for simulating matches. In addition to these weighted components, the algorithm applies an additive Elo-based adjustment on top of the base score to reflect a candidate's engagement history. The semantic similarity and profile alignment are derived from each user's profile data and the metadata for the project.

4.2.2 API

The matching engine is exposed through two layers. These layers are the FastAPI service that hosts the core Python matching logic and a Supabase Edge Function that ensures authentication is in place. Both endpoints require a valid Supabase JWT for authorization and allow for the dynamic tuning of weights to refine results. The request call for the project-to-person matching API takes a project object and a list of candidate profiles. The request body allows for optional weight customization and the exclusion of already-seen candidates from the results. The standard response includes a matches array, where each entry contains the target entity's details, the total score calculated, and an explanation object. The explanation field explicitly lists matched and missing skills to provide immediate feedback.

4.2.3 Implementation

The matching algorithm is implemented as a multi-stage pipeline in Python. It begins by computing semantic similarity between a user's profile text and project descriptions, which uses a pre-trained sentence transformer model (all-MiniLM-L6-v2) from the sentence-transformers library. It then calculates the number of matched and missing skills required for the project from a user's skill set. Additionally, the interest alignment is determined by measuring overlap between a user's interests and project tags. These three components are multiplied by their respective weights and summed to produce a base score, to which an additive Elo adjustment is then applied to determine the total match score for a given user. The final ranked list is then returned via the API, ensuring an explainable breakdown of matched skills and interests.

4.2.4 Potential Undesired Behaviours

The integration of an Elo-based reputation system, while beneficial for ensuring high-quality individuals are recommended first, includes the structural risk of "Cold Start" feedback loops. That is, new users will begin with a baseline Elo score, which inherently places them at a lower rank than established entities with a history of high engagement. As the Elo contribution is a bounded additive adjustment is held to small increments calculated using the relative mean and population mean of candidates in the current request. This prevents certain candidates from dominating the ranking based on its Elo and the semantic and skill-based components remain to be the greater driver for discovery by the algorithm. However, consistent negative engagement over time will still push a candidate's rating below the population mean producing a small persistent, yet desired, penalty. Profile sparsity could also potentially serve as an undesired behaviour if a user provides limited biographical information or

incomplete skill lists. In these cases, the semantic similarity component may lack sufficient context, forcing the algorithm to rely primarily on explicit skill overlap, which can reduce recommendation quality.

4.3 Person-to-Project Matching Algorithm

4.3.1 Normal Behaviour

Similar to the project-to-person matching algorithm, the person-to-project matching algorithm is responsible for helping users discover projects that align with their skills, interests, and experience. This algorithm operates from the perspective of a candidate user and prioritizes projects that are both technically suitable and contextually relevant to a given user.

4.3.2 API

The person-to-project matching functionality is exposed to the same two-layer architecture described in 4.2.2 and requires authentication via a valid Supabase JWT. This endpoint is called by the client application whenever a user accesses the project browsing feed. The request for the person-to-project matching API includes the authenticated user's profile and optionally allows customization of weights and the exclusion of already-seen projects from the results. The API will then respond with a ranked list of projects ordered by descending total match score. Each response entry includes the project's metadata, the computed total match score, and an explanation object. The explanation object explicitly identifies the matched and missing skills, matched interests, and the contribution of each scoring component, allowing developers working on the matching algorithm to understand why a project was recommended.

4.3.3 Implementation

This matching algorithm is implemented as a multi-stage pipeline in Python. For each project, the algorithm computes the semantic similarity between the users' profile text and the project description using a pre-trained sentence transformer model in which vector embeddings are generated for both inputs and compared using cosine similarity. It then calculates the number of matched and missing skills required for the project from a user's skill set. Additionally, the interest alignment is determined by measuring overlap between a user's interests and project tags. These components, along with the stored Elo of a given user, are multiplied by their respective weights and added to determine the total match score for a given user. The final ranked list is then returned via the API, ensuring an explainable breakdown of matched skills and interests.

4.3.4 Potential Undesired Behaviours

Similar to the project-to-person matching algorithm, a "cold start" feedback loop due to the Elo-based reputation system is also possible with this algorithm, though the same bounded additive design limits any undesired impacts from using the Elo system. Additionally, the aforementioned profile sparsity issue could also be applied to this algorithm (Potential Undesired Behaviours, 4.2.4).

4.4 Sign in and Sign up

4.4.1 Normal Behaviour

The authentication system in Peer.io is responsible for allowing users to create accounts, sign in, and securely access the application. Under normal behaviour, a new user provides an email address and

password to sign up, receives a verification email, and gains access only after confirming their email. Returning users can log in using their existing credentials. This ensures that only verified users are able to use the platform.

The system communicates with Supabase Auth through an API that exchanges data in JSON format over HTTPS. The primary inputs are the user's email and password, while the outputs include an authentication session token and a user object containing basic account information. During sign-up, Supabase also triggers an email confirmation process, and the client receives either a success response or an error message depending on the result of the request.

4.4.2 Implementation

The implementation consists of simple frontend sign-in and sign-up screens that call Supabase Auth methods to handle authentication. These components are responsible for collecting user input and forwarding it to the authentication service, while Supabase manages user records, sessions, and email verification internally.

4.4.3 Potential Undesired Behaviours

Potential undesired behaviours include users entering invalid credentials, failing to receive the verification email, attempting to log in before confirming their email, or encountering network-related errors during authentication. These cases are handled through error messages and restricted access.

4.5 Project Browsing Feed

4.5.1 Normal Behaviour

The Project Browsing Feed allows users who want to join new projects to find projects best suited to them. Filters will be present to allow for filtering of location and skills required. A project will appear on the screen, providing a brief description, the key skills required, and an image representing the project. Users browse through projects by either liking or disliking the project. This can be done through swiping left (to dislike) and swiping right (to like). A popup tutorial explaining this functionality will appear the first time a user sees the screen, with a button to reassess the tutorial at any time. If the user has scrolled through every project on the platform, a "reset" button will be present instead of a project, prompting the user with the choice of going through the projects again.

4.5.2 Implementation

The Project Browsing Feed will directly call the Project-to-Person API, displaying the projects recommended to the user by the algorithm in the order the algorithm provides. Liking a project will be tracked in the database, so matches can be made if the like is reciprocal.

4.5.3 Potential Undesired Behaviours

Projects must not be duplicated or seen multiple times unless the user has clicked the reset button. Liking a project must successfully send the like to the backend, as we do not want users' likes to get "lost".

4.6. Candidate Browsing Feed

4.6.1 Normal Behaviour

This page allows project owners to browse through potential candidates with skills suited to their project(s). In the top right, there is an option for users to add filters. These filters can include specific projects for which they want candidates, location of candidates, and skills. Users browse through candidates by either liking or disliking their profiles. This can be done through swiping left (to dislike) and swiping right (to like). Only one candidate is shown at a time.

4.6.2 Implementation

The Project Browsing Feed will directly call the Person-to-Project API, displaying the candidates recommended to the project owner by the algorithm in the order the algorithm provides. Liking a candidate will be tracked in the database so matches can be made if the like is reciprocal.

4.6.3 Potential Undesired Behaviours

Candidate profiles must not be duplicated or seen multiple times unless the user has clicked the reset button. Liking a candidate must successfully send the like to the backend, as we do not want users' likes to get "lost".

4.7 Matches Page

4.7.1 Normal Behaviour

The matches page displays all active matches available to the current user. It is split into two selectable tabs: Projects and Candidates. These feeds will populate when "Matches" occur. A match occurs when two conditions have been met: Firstly, the candidate user has liked the project upon seeing it in the Project Browsing Feed. Secondly, the project owner user has liked on the candidate user upon seeing them in the Candidate Browsing Feed. Once both conditions are met, the candidate user will see the project's name and picture under the project tab, and the project owner user will see the candidate's name and picture (as well as the applied project if the project owner owns more than one project) under the candidates tab. Tapping the project will enter the messaging system with the project owner, and tapping the candidate will enter the messaging system with the candidate user. Once a project is completed and has been closed by the project owner, the match will disappear, removing it from the matches page.

4.7.2 Implementation

On opening the matches page, the matches table in the database will be checked to determine profiles and projects to display.

4.7.3 Potential Undesired Behaviours

Under no circumstances should projects or users appear on the matches page that have not properly matched to avoid user confusion. For the same reason, once a project is closed, the respective project and candidates should not remain on the matches page.

4.8 Messaging System

4.8.1 Normal Behaviour

Users who have matched and appeared on the matches page will have a simple messaging window, where text can be exchanged between the users to facilitate project completion. The window will contain a text box, a fully scrollable history of the user's conversation, and the user's name and associated project in the top.

4.8.2 Implementation

Each conversation has a respective table in the database, and each message sent will add to a messages table associated with that table. Supabase Realtime will be utilized to ensure all updates to the messaging table are instantly relayed to the users in the chat, resulting in a seamless instant messaging experience.

4.8.3 Potential Undesired Behaviours

Messages must remain private and only be visible between the two users present in the conversation, as lack of privacy is undesirable.

4.9 Profile Page

4.9.1 Normal Behaviour

Users can edit their profile and account details. This includes updating their skills, skill level, bio, interests, education, work experience, and portfolio or changing account details such as displayed name, and location. This page also has access to the resume parsing functionality, allowing users to upload their resume and create an initial profile or add to their current profile.

4.9.2 Implementation

Every field in the profile page corresponds to an attribute in the *profiles* table. When a user edits these fields those changes are updated in the database and this new information will be used in following iterations of the matching algorithm. In addition, through this page, users can access the resume parser through an API endpoint as explained above.

4.9.3 Potential Undesired Behaviours

Changes made by the user must be reflected in the database immediately after editing. Delayed information changes would result in discrepancies between the profiles shown to project owners and the profile the user wishes to display. Ensuring the changes take effect immediately means other users only see the profile this user wishes to display. In addition, we want project suggestions to be made based on the most accurate and recent user data.

4.10 Project Creation and Editing

4.10.1 Normal Behaviour

The project components of the application are the interface through which project owners can create, manage, and edit their project listings. This component enables defining what their building is and what kind of collaborators they’ll be looking for.

4.10.2 Implementation

Users are able to create new projects with the following fields: title, description, skills needed, and tags. The skills needed will refer to the skills desired in collaborators, and will be utilized by our matching algorithm to pair them with individuals who suit the project's needs. The tags field under the project will help the application categorize projects so that the matching algorithm can suggest users who have mutual interests in the projects.

4.10.3 Potential Undesired Behaviours

The changes made to a project by the user must be reflected in the database immediately after editing. Delayed information changes can result in confusion during communication about projects. In addition, the project management interface should allow users to view all owned projects, edit projects, toggle the active status of their projects, and delete projects to ensure users feel in control of their projects. Lack of these features would result in lack of user control, an undesired outcome.

5. Backend

5.1 Intro

Supabase is used as the primary backend platform for Peer.io and provides a PostgreSQL-based database. In addition to database services, Supabase is used for authentication, edge functions, and object storage for files such as profile images and resumes. In addition to Supabase, Railway is used to host images of our Python algorithms. Namely, our matching algorithm and our resume parser.

5.2 Database Schema

The database schema shown in *Appendix, Image 1* illustrates the relationships between the core tables, including profiles, projects, conversations, conversation_participants, matches, candidate_likes, project_likes, messages, and resume_files.

In the schema diagram, primary keys are indicated by the key icon at the very left of the column name. Column nullability is represented by the diamond icon preceding the column name: a white diamond indicates a non-nullable column, while a black diamond indicates a nullable column. Each column’s data type is shown right-adjacent to its name. Lines connecting columns across tables represent foreign key relationships and define the links between entities.

The schema includes a unique entity, auth.users.id, which is managed by Supabase Auth. The profiles table is keyed by auth.users.id, linking authenticated users to their application-specific profile data. This allows profile information to evolve independently of authentication concerns.

5.3 Storage Buckets

We use Supabase Storage Buckets to store profile images and resumes. The profiles table includes profile_image and resume_url columns (type text), which store the storage paths to the corresponding files in Supabase Storage. These paths are used to generate public or signed URLs at runtime based on access permissions.

5.4 Edge Functions

We use Supabase Edge Functions to run backend logic close to the user without managing servers. These functions are deployed on Supabase's edge infrastructure and can securely access Supabase Auth, Storage, and Postgres using server-side credentials. Peer.io uses two Edge Functions: resume-parser and match-api. Both of which act as authentication proxies that validate the caller's Supabase JWT before forwarding their requests.

Resume-parser (section 4.1):
Endpoint: https://lrstnbamnilrjpevdjlm.supabase.co/functions/v1/resume-parser
Input: Digital or scanned PDF resume file or resume URL
Output: Structured resume data including extracted text, detected sections (Education, Experience, Projects, Skills), and normalized skill list

Match-api (section 4.2 and 4.3):
Endpoint: https://lrstnbamnilrjpevdjlm.supabase.co/functions/v1/match-api
The match-api function proxies requests to the FastAPI matching service and exposes two routes:

Match-candidates route (/match/candidates — section 4.2):
Input: project object (description, skills, tags), candidate profiles, optional weights, optional exclude_candidate_ids
Output: ranked_candidates array, count

Match-projects route (/match/score — section 4.3):
Input: user profile (bio, skills, interests), project list, optional weights, optional exclude_project_ids
Output: ranked_projects array, count

5.5 Railway

We use Railway to host the two Python backend services that the Supabase Edge Functions proxy to. Railway handles provisioning and exposes each service on a stable URL without requiring us to manage infrastructure. Railway hosts two different API endpoints, one for the entirety of the Matching Algorithm which handles person-to-project matching and project-to-person matching and one for handling the Resume Parser. Both have respective health endpoints to ensure each endpoint is up.

6. UI/UX Design

We chose a simplistic and intuitive design inspired by apps such as Hinge and Instagram. This includes a plain interface with few colours and simple fonts, allowing the candidate profiles and projects to take the spotlight.

The colours we include in our interface will be muted and calm, not bright and distracting, ensuring the app itself does not capture the user's attention. Our main palette will consist of calm greens. Colour will be used to draw a user's attention; however, backgrounds and non-crucial icons will be in

grey-scale. This allows for users' attention to be drawn to the projects and profiles, not our interface. For our font, we prioritized simplicity and readability, allowing for increased accessibility. Therefore, we chose the font *Inter* for both titles and paragraph text.

Every page includes the bottom navigation bar, similar to Instagram, to access the project feed, candidate feed, matches and account details. This allows for intuitive and familiar navigation to key pages. The initial draft can be found below. Our Figma prototype displaying user interactions and key pages is also accessible here.

See *Appendix, Image 2.*

7. Validation & Verification

Extensive manual / exploratory end-to-end testing was conducted on all aspects of the application by QA Lead Martin.

7.1 Matching

Tests will be implemented to ensure that a user can match to a project, a user can match to multiple projects, and that a user cannot match to closed projects. This will include a combination of basic unit tests, as well as end-to-end manual testing.

See Appendix, Table 2.

7.1.1 Unit Tests

Test cases will include simulating matches for a project and user, ensuring that all of the outputs of the matching algorithm are valid matches and in a logical order. These tests will ensure the weighted summation works, the skills are considered correctly, the elo rating doesn't prevent discoverability for a given project or user, and users and projects that lack a sufficient amount of description are still able to be matched to some extent.

The matching algorithm test suite (matching_algorithm/tests/test_matching.py) contains 73 unit tests organized across the following areas:

- **Skill Matching (4 tests):** Validates perfect skill match, partial skill overlap, no matching skills, and case-insensitive comparison between user profiles and project requirements.
- **Interest Matching (13 tests):** Tests interest overlap detection and handling of completely disjoint interest sets, case insensitivity, separator normalization (hyphens, underscores), stemmed variant matching (plural/singular, -ing forms), and neutral fallback behaviour when either side has no data.
- **Semantic Similarity (3 tests):** Verifies the NLP-based semantic similarity scoring between user descriptions and project descriptions, including similar text pairs, dissimilar text pairs, and empty input handling.
- **Weighted Scoring (3 tests):** Ensures the weighted summation of match components (semantic similarity, skill overlap, interest overlap) produces correct total scores, ranks projects in the expected order, and returns accurate score breakdowns.
- **Custom Weights (2 tests):** Tests that user-specified custom weights are applied correctly and that invalid weight configurations are rejected with appropriate errors.

- **Empty Input Edge Cases (9 tests):** Covers scenarios where users or projects have missing or empty fields (no skills, no description, no interests), ensuring the algorithm handles incomplete data gracefully without errors.
- **Score Endpoint Empty Guards (5 tests):** Validates that the /match/score endpoint handles empty or malformed inputs correctly at the API level.
- **Candidates Endpoint Empty Guards (5 tests):** Validates that the /match/candidates endpoint handles empty or malformed inputs correctly at the API level.
- **Elo System (29 tests):** Validates the ELO rating module and its integration with the matching engine, including score adjustment direction and magnitude, sigmoid clamping to increments of max_boost, population-mean-relative adjustments, reaction outcomes (like, pass, super_like), disabled-mode behaviour, custom K-factor and max_boost configuration, total score clamping to [0, 1], and the /elo/update API endpoint responses.

7.1.2 API Integration Tests

The matching API integration is tested from both the server side and the client side:

Server-side API tests (matching_algorithm/tests/test_api.py, 4 tests) verify the FastAPI endpoints directly: - GET /match/health health check returns expected status. - POST /match/score returns ranked project matches with valid scores. - POST /match/candidates returns ranked candidate matches. - POST /match/batch handles batch matching of multiple user-project combinations.

Client-side API integration tests (peerio_app/Tests/matching-api.test.ts, 14 tests) verify the mobile app's integration with the matching API: - getMatchedProjects (6 tests) correctly calls the API, transforms response data, handles fallback field names, handles errors and empty inputs, constructs a bio proxy when the user has no bio, and includes the Authorization header when a session exists. - getMatchedCandidates (3 tests) correctly fetches and transforms candidate ranking results, handles empty candidate lists, and handles error responses. - checkMatchingAPIHealth (3 tests) validates health check calls and error handling, including fetch-level failures. - getBatchMatchedProjects (2 tests) tests batch matching response transformation and error handling.

7.2 Messaging

Basic unit tests will be done, including sending and receiving both simple and long messages. Tests will ensure that messages are only enabled between matched users. This will also be tested for privacy, ensuring users external to the match cannot view other users' conversations. Messages should be near instant, so manual testing to validate messages received within 200ms will be done.

Additionally, time formatting utilities used in the messaging interface are tested (peerio_app/Tests/format-time.test.ts, 12 tests): - formatMessageTime correctly displays timestamps as “now”, “Xm ago”, “Xh ago”, time-of-day, or full date depending on recency. - formatRelativeDate correctly formats relative dates including “Today”, “Yesterday”, weekday names, and full date strings. - Tests use deterministic fake timers to ensure consistent behavior across environments.

7.3 Project Feed

Rigorous testing will be done to ensure that users’ likes and dislikes are properly recorded in the backend. UI elements will be tested manually, ensuring no visual or usability issues with the swiping

or buttons for liking a project. Basic unit tests will ensure the project feed is properly attached to the API, receiving and displaying the project recommendations in the order provided by the algorithm.

The project feed test suite (peerio_app/Tests/projects.test.ts, 17 tests) covers: - **Project Liking (likeProject):** Validates that liking a project correctly records the interaction in the project_likes table via Supabase, and handles database errors gracefully. - **Coordinate Fetching (fetchCoords):** Tests retrieval of project geographic coordinates for distance-based features, including error handling when coordinates are unavailable. - **Already-Swiped Filtering (fetchSwipedProjectIds):** Ensures that projects the user has already swiped on are correctly retrieved for filtering from the feed. - **Non-Match Like Cleanup (deleteNonMatchedProjectLikes):** Verifies that project likes that did not result in a match are properly cleaned up from the database. - **Project Fetching and Transformation (fetchProjects):** Tests the full pipeline of fetching projects from the database, transforming the data into the expected UI format, and handling various data shapes (object vs. array profile data).

7.4 Candidate Feed

Thorough testing will be done to ensure that project owners' likes and dislikes are properly recorded in the backend. UI elements will be tested manually, ensuring no visual or usability issues with the swiping or buttons for liking a candidate. Basic unit tests will ensure the candidate feed is properly attached to the API, receiving and displaying the candidate profile recommendations in the order provided by the algorithm.

The candidate feed test suite (peerio_app/Tests/candidates.test.ts, 33 tests) covers: - **Distance Calculations (calcDist):**Validates the Haversine formula implementation for calculating distances between geographic coordinates, tested with real-world city pairs (e.g., Toronto to Vancouver). - **Coordinate Fetching (fetchCoords, fetchMyCoords):** Tests retrieval of candidate and user geographic coordinates from the database, including error handling. - **Candidate Liking (likeCandidate):** Validates that liking a candidate correctly records the interaction in the candidate_likes table via Supabase, and handles database errors gracefully. - **Already-Swiped Filtering (fetchSwipedCandidateIds):** Ensures that candidates the user has already swiped on are correctly retrieved for feed filtering. - **Non-Match Like Cleanup (deleteNonMatchedCandidateLikes):** Verifies that candidate likes that did not result in a match are properly cleaned up from the database. - **Candidate Fetching and Transformation (fetchCandidates):** Tests the full pipeline of fetching candidates from the database, transforming the data, and filtering out already-swiped candidates. - **Project Ownership (fetchMyProjects):** Tests retrieval of the current user's projects for the candidate browsing context.

7.5 Profile Page

Testing will ensure that changes are saved to the database immediately after being entered by the user. This will be done through a combination of basic unit tests and end-to-end manual testing. Tests will also ensure that following uses of both the project-to-person and person-to-project matching algorithms will use this new information, again through basic unit tests and end-to-end manual testing. Specifically, we will be focusing on the edge cases where the algorithm is run while the user is editing their profile, and within milliseconds directly after. Finally, the UI will be manually tested to ensure a clear interface with no disruptions while inputting information and switching between fields.

The profile test suites cover database operations and URL handling:

User Profile Database Operations (peerio_app/Tests/user-profile.test.ts, 9 tests): - **fetchUserProfile:** Tests fetching a profile by user ID from the profiles table, including successful retrieval and error handling (returns null on Supabase error). - **getUserProfile:** Tests the higher-level profile retrieval

function, including fallback to the authenticated user's ID when no explicit user ID is provided, and fallback to a mock profile when no profile is found in the database. - **updateUserProfile:**Tests profile updates via Supabase, verifying that updated data is returned on success and null is returned on error.

Profile Image URL Resolution (peerio_app/Tests/profile.test.ts, 9 tests): - Tests resolution of absolute URLs (https://), protocol-relative URLs (//), and relative Supabase storage paths. - Validates fallback behavior when the image URL is null, undefined, or empty. - Ensures special characters in storage paths are correctly encoded.

7.6 Login

We will check the following manually to verify that our login system works as intended:

See Appendix, Table 3.

Pass criteria: For each case, the observed UI behaviour matches expected output from the specified actions/input, and no crashes occur.

7.7 Backend

We will check the following manually to verify that our backend system works as intended.

See Appendix, Table 4.

These 3 tests in the table mentioned above cover the majority of every aspect of our backend system, Authentication, Database, Buckets, and Edge Functions.

Pass criteria: For each case, the observed UI behaviour matches expected output from the specified actions/input, and no crashes occur.

7.7.1 Match Database Query Tests

In addition to the manual backend tests, automated unit tests validate the match database query layer (peerio_app/Tests/match-queries.test.ts, 13 tests):

- **fetchProjectMatchesOptimized (5 tests):** Tests fetching project matches for a candidate using optimized JOIN queries. Validates correct MatchUI array construction from joined data (matches + projects + owner/candidate profiles), graceful handling of Supabase PGRST116 "not found" errors, handling of other database errors, null data responses, and fallback values ("0") when nested profile data (projects, owner_profile, candidate_profile) is null.
- **fetchCandidateMatchesOptimized (2 tests):** Tests fetching candidate matches for a project owner, verifying correct MatchUI array construction and empty array return on database errors.
- **fetchAllMatchesOptimized (2 tests):** Tests the combined fetch of both project matches and candidate matches via Promise.all, including conversation participant enrichment. Validates that both projectMatches and candidateMatchesarrays are returned correctly, and handles the case where no matches exist.

7.8 Resume Parser

7.8.1 Unit Tests

The main goal of the resume parser unit tests is to verify reliable normalization, validation, and URL resolution logic within the client-side resume parsing library. Tests are implemented in TypeScript using the Jest framework and cover four core functions exported from lib/resume-parser.

The createEmptyParsedData tests confirm that the function returns a fully initialized ParsedData object with all fields set to their zero values, and that each call produces an independent copy.

The hasParsedResumeData tests verify that the function correctly identifies non-empty resume data. It returns false only when all fields are empty, and true as soon as any field (bio, location, any link, skills, interests, education, experience, or personal projects) contains a value.

The normalizeParsedResumePayload tests form the bulk of the suite and cover a wide range of inputs. They verify that null, undefined, and empty objects are handled gracefully by returning an empty ParsedData object. String fields such as bio and location are trimmed of whitespace and non-string values are coerced to empty strings. Link normalization is tested extensively: URLs that already contain a protocol are left unchanged, bare domains and www. prefixes have https:// prepended, and email addresses or plain text strings are left as-is. Array fields such as skills and interests are tested for case-insensitive deduplication, whitespace trimming, non-string filtering, and entry caps (30 for skills, 20 for interests). Structured entries for education, experience, and personal projects are tested for field trimming, auto-generated ID prefixes (edu-, exp-, proj-), filtering of fully empty entries, and link normalization on project URLs. A full, realistic resume payload test ties all of these together.

The getResumeParserUrl tests verify the URL resolution priority chain using a mocked expo-constants module and per-test environment variable overrides. The function prefers EXPO_PUBLIC_PARSER_EDGE_URL, then expoConfig.extra.parserUrl, then EXPO_PUBLIC_PARSER_URL, then constructs a URL from EXPO_PUBLIC_SUPABASE_URL, and finally falls back to DEFAULT_RESUME_PARSER_URL. Trailing slashes are stripped from all resolved URLs, and empty string environment variables are treated as absent.

See Appendix, Table 5.

7.8.2 Performance Test and Metrics

For the resume parser, the most important performance metric is end-to-end parsing latency. For typical digital PDF resumes, the system should return parsed results within 2 seconds under normal network conditions. Alternatively, for resumes requiring OCR, a maximum acceptable latency of 15 seconds is defined due to the higher computational cost of image processing.

In addition to latency metrics, reliability is measured as the fraction of resumes that return structured output without crashing, with a target success rate of at least 95%. Extraction quality is evaluated using a small, labelled test set by measuring the accuracy of section detection and the F1-score of skill extraction, with minimum targets of 0.85 for section detection accuracy and 0.70 for the skill extraction F1-score.

These metrics provide quantitative evidence that the Resume Parser meets both functional correctness and performance requirements for deployment.

See *Appendix, Table 6.*

7.9 Test Summary

The following table summarizes the automated test coverage across all system components:

Test File	Component	Test Count	Focus
matching_algorithm/tests/test_matching.py	Matching Engine	33	Skill/interest matching, semantic similarity, weighted scoring, edge cases
matching_algorithm/tests/test_api.py	Matching API (Server)	4	API endpoint validation (health, score, candidates, batch)
peerio_app/Tests/candidates.test.ts	Candidate Feed	33	Distance calc, liking, filtering, fetching, data transformation
peerio_app/Tests/projects.test.ts	Project Feed	17	Liking, coordinate fetching, filtering, data transformation
peerio_app/Tests/matching-api.test.ts	Matching API (Client)	16	API integration, response handling, auth headers, batch matching
peerio_app/Tests/match-queries.test.ts	Match Database Queries	13	Optimized JOIN queries, error handling, null fallbacks
peerio_app/Tests/format-time.test.ts	Messaging Utilities	12	Timestamp formatting, relative dates
peerio_app/Tests/user-profile.test.ts	Profile Database	9	Profile CRUD, auth fallback, mock profile fallback
peerio_app/Tests/profile.test.ts	Profile Image	9	URL resolution, storage paths, encoding

Total		**146**	

Appendix

Table 1:

System Component	Requirement(s) Covered
Resume Parser	● P0: User can upload their resume which will be parsed to fill in their initial profile
Project-to-Person Matching Algorithm + Project-to-Person Matching Algorithm	● P0: Matching projects/profiles based on the highest relevance score using an NLP-based semantic similarity, with explainable scoring and tunable weights ● P0: Elo matching system for profiles and projects ● P0: The client will reach out to the server for profile/project recommendations, the server will compute these recommendations using the matching algorithm and available profiles/projects from the database. ● P3: Incorporating previous matches, posts, inactivity and more into the matching algorithm
UI: Sign In and Sign Up	● P0: Account sign-up and sign-in pages ● P0: Resume parsing page for users to upload their resume ● P3: Upload your resume to auto-fill your profile
UI: Project Browsing Feed	● P0: Project browsing feed to look through the various projects available for collaboration ● P0: Post and profile interactions ● P1: Filtering on projects and people
UI: Candidate Browsing Feed	● P0: People browsing feed to look through people who want to collaborate ● P0:Post and profile interactions ● P1: Filtering on projects and people ● P2: Tag candidates with which of your projects they would best fit
UI: Matches	● P0: Matches page to see the people and projects you have matched with, and their contact information
UI: Profile and Editing	● P1: Profile and project customization ● P1: Ability to delete or pause (temporarily remove from matching feeds) your project or profile
UI: Project Creation	● P0: Project creation page to post a short description of your project

and Editing	idea ● P1: Profile and project customization ● P1: Allow multiple projects per person ● P1: Ability to delete or pause (temporarily remove from matching feeds) your project or profile ● P3: Ability to add a project to your archive, a set of projects a user has previously worked on and found through Peer.io
Messaging	● P1: Messaging system to communicate after matching
Database	● P0: Database setup to store project and user profile data ● P0: APIs for the client-side application to retrieve and edit their profiles and projects ● P0: APIs for the server to access profiles and projects

Table 2:

Test Case	Actions/Input	Expected Output	Result
Weighted summation	1. Provided a user and a small array of projects, run the matching algorithm to determine the total match score for each 2. Cross-reference the matches list, ordered by match score, to ensure the matching algorithm worked correctly.	The matching algorithm calculates the weighted summation of match scores to follow the desired logic.	PASS
Elo reputation	1. Create two identical profiles, and a different profile that provides a better match for the given project. Assign one of the identical profiles a higher Elo rating than the others, and assign the different profile a lower Elo rating. 2. Run the matching algorithm	The identical profiles are ranked according to Elo, while the better-suited profile is ranked highest despite the low Elo.	PASS

Insufficient descriptions	1. Create a profile that doesn't contain a sufficient amount of information for the number of interests, and a project that doesn't have a sufficient description 2. Run the matching algorithm	The algorithm handles the empty information fields without returning any errors and provides matches accordingly.	PASS
Matching + Animation	1. On two devices, have a user and a project like each other 2. Ensure Match animation plays and chat between the two people opens in Matches tab	Chat should be opened, match animation should play. Chat should be usable.	PASS

Table 3:

Test Case	Actions/Input	Expected Output	Result
Valid sign-in	3. Enter valid email and valid password 4. Click the Sign In button	User is redirected to the app/home screen and session persists after app/refresh/reopen	PASS
Invalid credentials	3. Enter valid email and wrong password 4. Click the Sign In button	Show error message and remain on login screen	PASS
Invalid email format	1. Enter "abc" as email 2. Click the Sign In button	Show error message and remain on login screen	PASS
Unverified email	1. Sign up for a new account with an email and password 2. Before verifying the confirmation email, click the Sign In button	The app blocks sign in due to unverified email, show error message and remain on login screen	PASS

Table 4:

Test Case	Actions/Input	Expected Output
Authentication + Profile Creation (Create a new user account and verify that authentication and database linkage work correctly)	1. Sign up with a new email and password. 2. Verify the email and sign in. 3. Check that a corresponding row exists in the profiles table with profiles.id = auth.users.id.	The user can authenticate successfully, and profile data is correctly linked to the authenticated user.
File Upload + Storage Integration (Verify that file storage and database references are correctly handled)	1. Upload a profile image or resume. 2. Confirm the file appears in the correct Supabase Storage bucket. 3. Verify that profile_image or resume_url in the profiles table stores the correct storage path.	The file is accessible through a generated URL, and the database reference correctly points to the stored file.
Project Matching via Feed (Verify that the project matching Edge Function is executed correctly and returns relevant results)	1. Log in as a user with defined skills and interests stored in their profile. 2. Navigate to the projects feed screen. 3. Observe the list of projects returned.	The feed displays projects that align with the user's skills and interests, confirming that the match-projects Edge Function executes successfully and returns relevant matches.

Table 5:

Test Case	Actions/Input	Expected Output
Returns fully empty ParsedData object	Call createEmptyParsedData()	Object with bio: '', location: '', empty links, and empty arrays for all list fields
Returns new object on each call	Call createEmptyParsedData() twice, mutate first result	Second result is unaffected by mutations to the first

Returns independent links objects	Call createEmptyParsedData() twice, set links.github on first	Second result's links.github remains ''
Returns false for fully empty data	Call hasParsedResumeData(createEmptyParsedData())	false
Returns true when bio is set	Set bio = 'Hello', call hasParsedResumeData	true
Returns true when location is set	Set location = 'Toronto', call hasParsedResumeData	true
Returns true when any link is set	Set links.github = 'https://github.com/user', call hasParsedResumeData	true
Returns true when skills are present	Set skills = ['React'], call hasParsedResumeData	true
Returns true when interests are present	Set interests = ['AI'], call hasParsedResumeData	true
Returns true when education is present	Set one education entry, call hasParsedResumeData	true
Returns true when experience is present	Set one experience entry, call hasParsedResumeData	true
Returns true when personal_projects is present	Set one project entry, call hasParsedResumeData	true
Handles null input	normalizeParsedResumePayload(null)	Empty ParsedData object
Handles undefined input	normalizeParsedResumePayload(undefined)	Empty ParsedData object
Handles empty object input	normalizeParsedResumePayload({})	Empty ParsedData object
Trims whitespace from bio and location	{ bio: ' hello ', location: ' Toronto ' }	bio: 'hello', location: 'Toronto'
Returns empty string for non-string bio	{ bio: 42 }, { bio: null }, { bio: true }	bio: ''
Preserves URLs with existing protocol	{ links: { github: 'https://github.com/user' } }	links.github: 'https://github.com/user'
Prepends https:// to www. links	{ links: { portfolio: 'www.example.com' } }	links.portfolio: 'https://www.example.com'
Prepends https:// to bare github.com links	{ links: { github: 'github.com/user' } }	links.github: 'https://github.com/user'

Prepends https:// to bare linkedin.com links	{ links: { linkedin: 'linkedin.com/in/user' } }	links.linkedin: 'https://linkedin.com/in/user'
Prepends https:// to bare twitter.com links	{ links: { twitter: 'twitter.com/user' } }	links.twitter: 'https://twitter.com/user'
Prepends https:// to bare x.com links	{ links: { twitter: 'x.com/user' } }	links.twitter: 'https://x.com/user'
Prepends https:// to bare instagram.com links	{ links: { instagram: 'instagram.com/user' } }	links.instagram: 'https://instagram.com/user'
Prepends https:// to domain-like strings	{ links: { portfolio: 'mysite.io' } }	links.portfolio: 'https://mysite.io'
Does not prepend https:// to email-like strings	{ links: { other: 'user@example.com' } }	links.other: 'user@example.com'
Does not prepend https:// to plain text with spaces	{ links: { other: 'not a link' } }	links.other: 'not a link'
Returns empty string for missing links	{ links: {} }	links.github: '', links.linkedin: ''
Deduplicates skills case-insensitively	{ skills: ['React', 'react', 'REACT', 'TypeScript'] }	['React', 'TypeScript']
Filters non-string and empty skill entries	{ skills: ['React', null, 42, '', ' ', 'Go'] }	['React', 'Go']
Trims skill strings	{ skills: [' React ', ' Go '] }	['React', 'Go']
Caps skills at 30 entries	Array of 40 unique skill strings	Array of length 30
Returns empty array when skills is not an array	{ skills: 'React' }, { skills: null }	[]
Deduplicates interests case-insensitively	{ interests: ['AI', 'ai', 'ML'] }	['AI', 'ML']
Caps interests at 20 entries	Array of 25 unique interest strings	Array of length 20
Maps education entries correctly	{ education: [{ id: 'edu-1', school: 'MIT', degree: 'BSc CS', year: '2024' }] }	Entry with matching fields
Generates id when education entry has no id	{ education: [{ school: 'MIT', degree: 'BSc', year: '2023' }] }	id matches /^edu-/
Filters out education entries with no meaningful fields	Entry with school: '', degree: '', year: ''	Entry excluded from results
Trims whitespace from education fields	{ school: ' MIT ', degree: ' BSc ', year: ' 2024 ' }	Fields trimmed to 'MIT', 'BSc', '2024'

Returns empty array when education is not an array	{ education: null }, { education: 'MIT' }	[]
Maps experience entries correctly	{ experience: [{ id: 'exp-1', company: 'Acme', position: 'Engineer', duration: '2yr', description: 'Built stuff' }] }	Entry with matching fields
Generates id when experience entry has no id	{ experience: [{ company: 'Acme', position: 'Dev', duration: '1yr', description: '' }] }	id matches /^exp-/
Filters out experience entries with no meaningful fields	Entry with all empty fields	Entry excluded from results
Keeps experience entries that have only a description	{ company: '', position: '', duration: '', description: 'Did things' }	Entry included in results
Returns empty array when experience is not an array	{ experience: null }	[]
Maps personal_projects entries correctly	{ personal_projects: [{ id: 'proj-1', name: 'App', description: 'Cool app', link: 'https://app.com' }] }	Entry with matching fields
Generates id when project entry has no id	{ personal_projects: [{ name: 'App', description: '', link: '' }] }	id matches /^proj-/
Normalizes project link	{ personal_projects: [{ name: 'App', link: 'github.com/user/app' }] }	link: 'https://github.com/user/app'
Filters out project entries with no meaningful fields	Entry with name: '', description: '', link: ''	Entry excluded from results
Returns empty array when personal_projects is not an array	{ personal_projects: null }	[]
Handles a realistic full resume payload	Full payload with all fields, bare URLs, and null values	All fields normalized, links prefixed with https://, arrays populated correctly
Generates unique ids across all entry types	Two education, two experience, two project entries all without ids	All 6 generated ids are distinct with correct prefixes
Returns default URL when no env vars or config set	No env vars set, no expoConfig	DEFAULT_RESUME_PARSER_URL
Prefers EXPO_PUBLIC_PARSER_EDGE_URL over all others	All env vars and expoConfig set	Returns normalized EXPO_PUBLIC_PARSER_EDGE_URL

Falls back to expoConfig.extra.parserUrl	Only expoConfig parserUrl set	Returns normalized parserUrl from config
Falls back to EXPO_PUBLIC_PARSER_URL	Only EXPO_PUBLIC_PARSER_URL set	Returns normalized EXPO_PUBLIC_PARSER_URL
Constructs URL from EXPO_PUBLIC_SUPABASE_URL	Only EXPO_PUBLIC_SUPABASE_URL set	Returns <supabase_url>/functions/v1/resume-parser
Strips trailing slash from resolved URL	EXPO_PUBLIC_PARSER_EDGE_URL = 'https://edge.example.com/path/'	'https://edge.example.com/path'
Returns default URL for empty string env vars	All env vars set to ''	DEFAULT_RESUME_PARSER_URL

Table 6:

Test case	Actions/Input	Expected Output
Digital resume latency	Parse 30 digital PDF resumes	latency ≤ 2 seconds
OCR resume latency	Parse 30 scanned resumes	latency ≤ 15 seconds
System reliability	Parse mixed dataset	≥ 95% of inputs return structured output successfully
Section detection accuracy	Evaluate on labeled test set	Section detection accuracy ≥ 0.85
Large PDF handling	Upload high-page-count PDF	Completes within timeout or returns controlled error (slightly higher latency accepted under this circumstance)

Image 1:

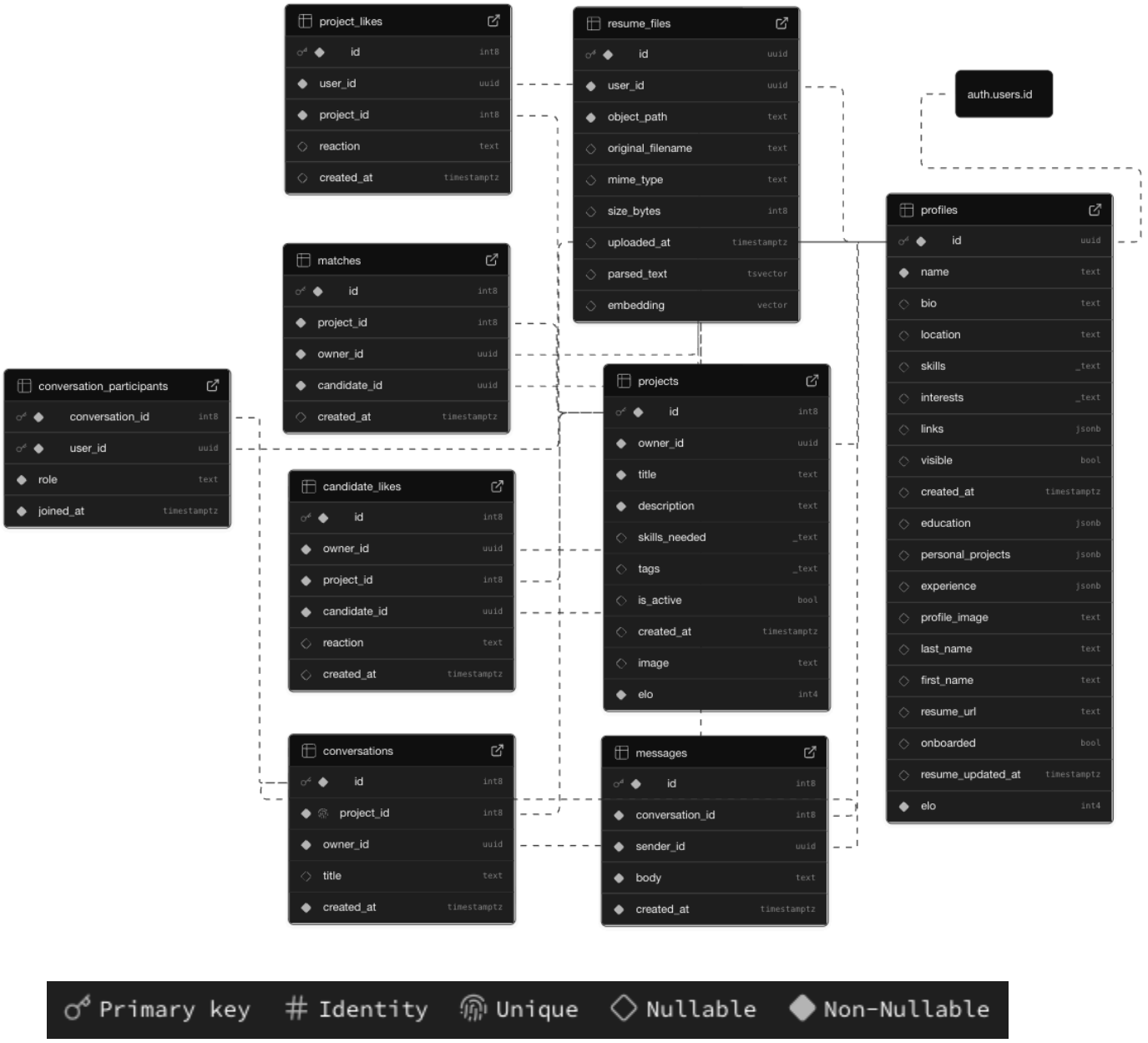
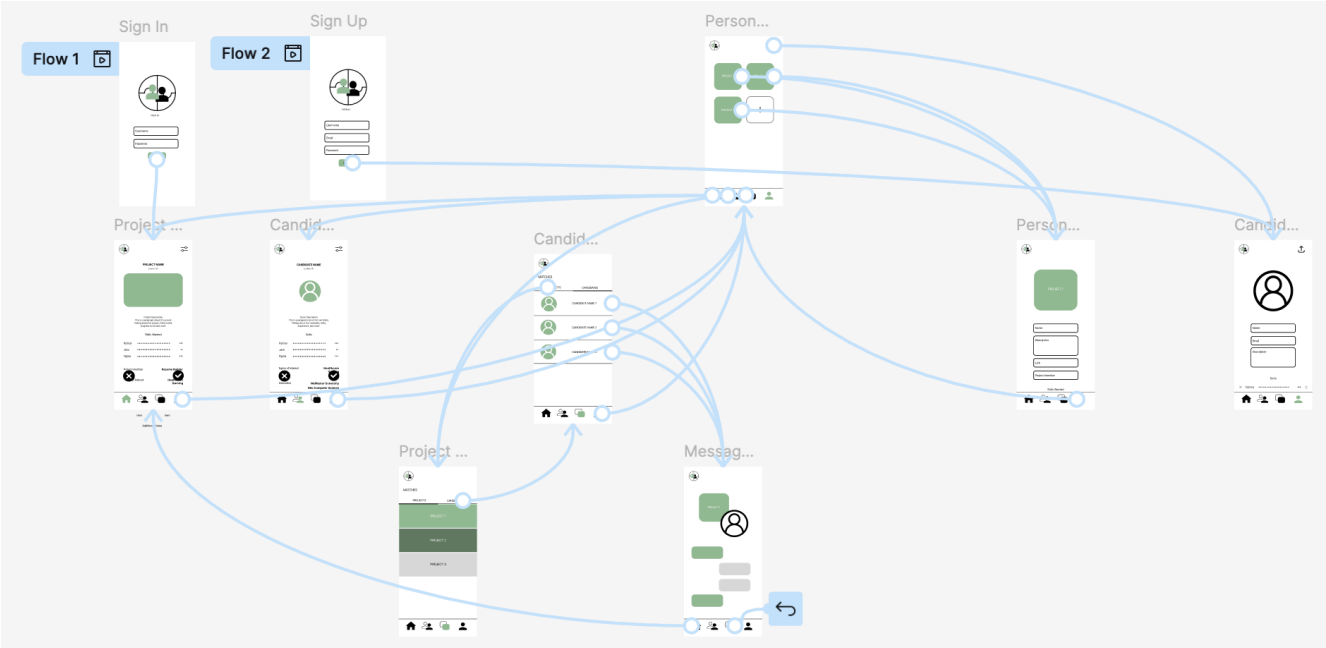

project_likes
id int8
user_id uuid
project_id int8
reaction text
created_at timestamptz
resume_files
id uuid
user_id uuid
object_path text
original_filename text
mime_type text
size_bytes int8
uploaded_at timestamptz
parsed_text tsvector
embedding vector
auth.users.id
profiles
id uuid
name text
bio text
location text
skills _text
interests _text
links jsonb
visible bool
created_at timestamptz
education jsonb
personal_projects jsonb
experience jsonb
profile_image text
last_name text
first_name text
resume_url text
onboarded bool
resume_updated_at timestamptz
elo int4
matches
id int8
project_id int8
owner_id uuid
candidate_id uuid
created_at timestamptz
conversation_participants
conversation_id int8
user_id uuid
role text
joined_at timestamptz
projects
id int8
owner_id uuid
title text
description text
skills_needed _text
tags _text
is_active bool
created_at timestamptz
image text
elo int4
candidate_likes
id int8
owner_id uuid
project_id int8
candidate_id uuid
reaction text
created_at timestamptz
conversations
id int8
project_id int8
owner_id uuid
title text
created_at timestamptz
messages
id int8
conversation_id int8
sender_id uuid
body text
created_at timestamptz
Primary key
Identity
Unique
Nullable
Non-Nullable

Image 2:


Flow 1
Sign In
Flow 2
Sign Up
Person...
Project ...
Candid...
Candid...
Person...
Candid...
Project ...
Messag...